

MAIE 6000C: Engineering AI Products: From Prototype to Production

Week 4 — Lab

Program: MSc in AI & Entrepreneurship

Term: Fall 2026/27

Instructor: Lfteris NTAFL0S, PhD

The Hong Kong University of Science and Technology

Today's Rule: Observe Before You Design

Before proposing a system, understand one real workflow



Question for today:

What must the system remember at each step?

Live Demo: What We Will Observe

We will observe:

- POST /cases creates a case and job
- PostgreSQL stores both records
- Case starts as queued
- Job starts as pending
- Worker claims and processes the job
- Worker calls the internal AI service
- Worker persists AI output
- API returns the final case state
- Structured logs explain progression

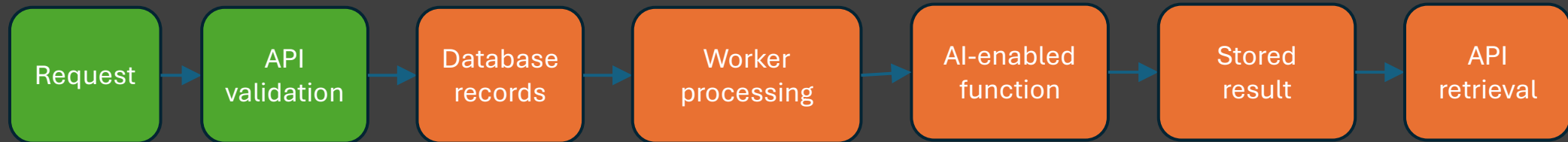
```
# 1. Open docker desktop
# 2. Open Powershell / Bash Terminal
# 3. cd into working folder
docker compose up --build -d
docker compose ps
docker compose stop worker

# can also keep API Swagger page open:
http://localhost:8000/docs
```

Live Demo Step 1: Check Service Health

```
# PowerShell
docker compose ps
curl.exe http://localhost:8000/health/live
curl.exe http://localhost:8000/health/ready
# Bash: use curl (without .exe)
```

Live Demo Step 2: Create a Case Through the API



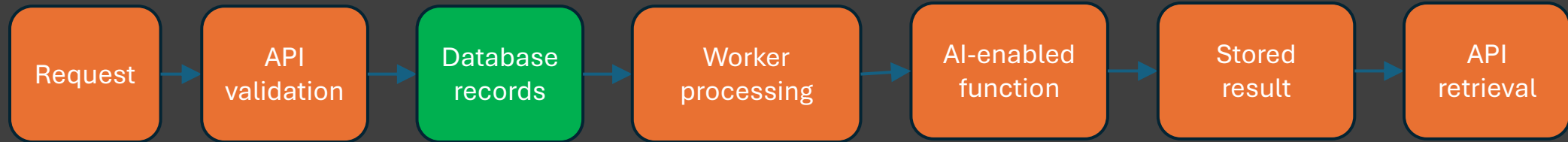
PowerShell

```
Invoke-RestMethod -Uri "http://localhost:8000/cases" -Method Post -ContentType "application/json" -Body '{"title":"Production outage alert","description":"Several users report the service is down and requests are failing."}' | ConvertTo-Json -Depth 10
```

Bash

```
curl -s -X POST "http://localhost:8000/cases" -H "Content-Type: application/json" -d '{"title":"Production outage alert","description":"Several users report the service is down and requests are failing."}' | python -m json.tool
```

Live Demo Step 3: Inspect the Persisted Records



```
# PowerShell or Bash
```

```
# cases table
```

```
docker compose exec db psql -U postgres -d maie6000c -c "SELECT id, title, status, ai_label,  
created_at FROM cases ORDER BY created_at DESC LIMIT 3;"
```

```
# jobs table
```

```
docker compose exec db psql -U postgres -d maie6000c -c "SELECT id, case_id, job_type, status,  
attempts, error FROM jobs ORDER BY created_at DESC LIMIT 3;"
```

db: The name of the specific service (container) defined in your docker-compose.yml

psql: Launches the PostgreSQL interactive terminal command-line tool inside that container.

-U postgres: Connects to the database using the postgres username (the default administrative user).

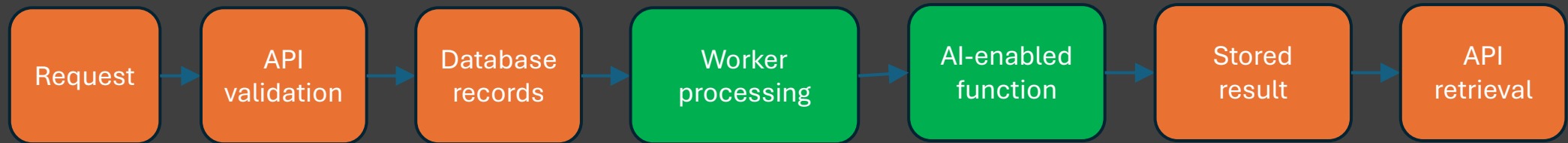
-d maie6000c: Specifies that you want to connect to the database named maie6000c.

-c "...": Tells psql to run the specific SQL command inside the quotation marks

Live Demo Step 4: Retrieve Current State Through the API

```
# PowerShell or Bash  
curl http://localhost:8000/cases/CASE_ID
```

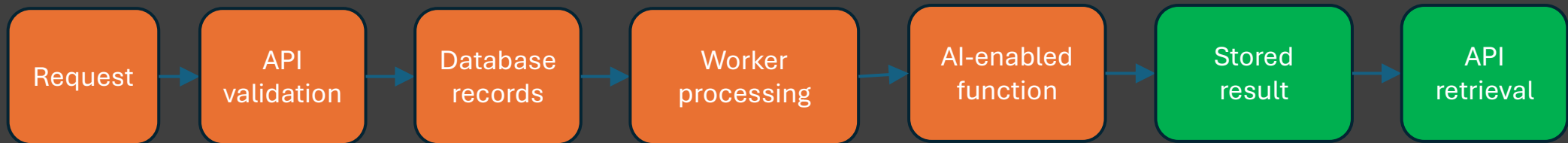
Live Demo Step 5: Start the Worker and Observe Processing



```
# PowerShell or Bash - Terminal 1
docker compose start worker
docker compose logs -f worker
```

```
# PowerShell or Bash - Terminal 2
docker compose logs --tail=20 ai
```

Live Demo Step 6: Inspect the Final Persisted State



```
# PowerShell (for Bash use curl without .exe)
curl.exe http://localhost:8000/cases/CASE_ID
curl.exe http://localhost:8000/jobs/JOB_ID
```

```
# PowerShell or Bash
```

```
# cases table
```

```
docker compose exec db psql -U postgres -d maie6000c -c "SELECT id, title, status, ai_label, created_at FROM cases ORDER BY created_at DESC LIMIT 3;"
```

```
# jobs table
```

```
docker compose exec db psql -U postgres -d maie6000c -c "SELECT id, case_id, job_type, status, attempts, error FROM jobs ORDER BY created_at DESC LIMIT 3;"
```

Live Demo Debrief: What Was Persisted, Logged, and Derived?

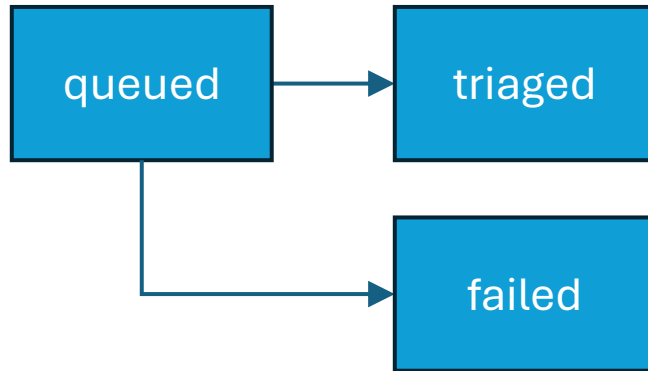
Category	What we observed
Persisted	case, job, statuses, attempts, timestamps, AI output
Logged	request completed, job claimed, job completed, service events
Derived	count of triaged cases, average processing time, failure rate

Key distinction

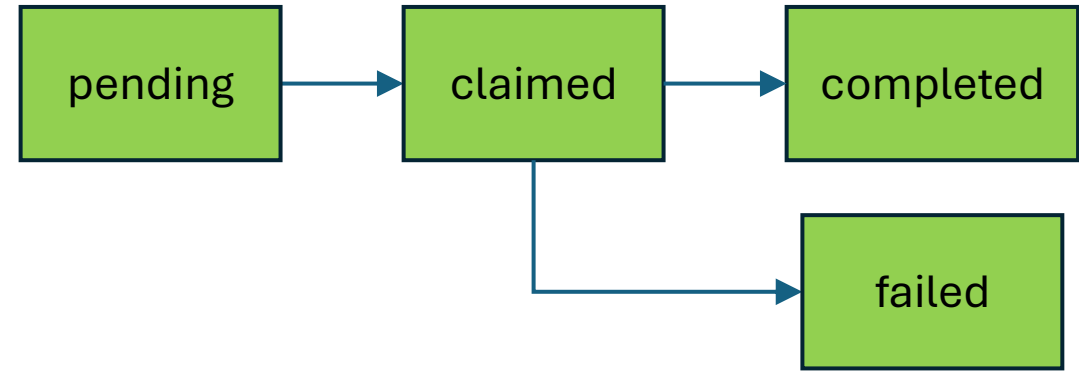
- Logs explain execution.
- Database records represent system truth over time.

Live Demo Debrief: Current State Transitions

Case

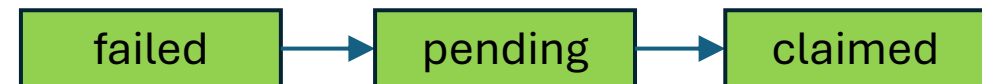


Job



Discussion question

- What would be required to support:



Activity A: Build a Persistence Evidence Map

Question	Evidence:
Which API route starts the workflow?	
Which function creates the case and job?	
Which model defines the foreign key?	
Which function claims work?	
Which function records failure?	
Where is AI output persisted?	
Which test verifies creation?	
Which smoke test verifies end-to-end processing?	

- **Team activity — 8 minutes**
- Open the starter repository and complete this table.
- **Do not write vague answers. Use paths and function names.**

Models, Migrations, and Tests Are Different Layers

- **Key rule**
- **Changing a Python model is not enough.**
The database structure must evolve through a reviewed migration.

Layer	Purpose
SQLAlchemy model	Python representation of desired tables
Alembic migration	Versioned change to real database structure
API/worker code	Reads and writes records
Tests	Verify claimed behavior
CI	Applies migrations and runs checks automatically

+

•

○

Activity B: Migration and Schema Audit

Team activity — 7 minutes

Inspect:

```
services/common/models.py  
alembic/versions/20260713_0001_create_initial_tables.py  
tests/integration/test_api_case_flow.py
```

Answer:

1. What foreign key enforces the case–job relationship?
2. Which columns support auditability or diagnosis?
3. Which columns represent current state?
4. What future project change would require a migration?
5. How might existing data be affected?

Activity C: Baseline-to-Proposal Gap Analysis

Team activity — 7 minutes

Keep from starter	Adapt or replace	Add only if needed	Open risk/question
API → DB → worker pattern	case fields and names	review decision table	data quality
health checks	AI logic	event history	external-service reliability
migrations	job type	result table	project scope

Session 1 Checkpoint

Before Session 2, each team should have

- ✓ a completed persistence evidence map;
- ✓ one observed starter state-transition diagram;
- ✓ one migration-related observation;
- ✓ a keep / adapt / add / risk list;
- ✓ one clearly stated question for the team proposal.

Save your notes

- Use a shared team working note or begin updating:
 - docs/architecture.md



Session 2: Turn Evidence into a Buildable Plan

From starter evidence to your team architecture

Each proposal should answer:

- What user workflow are we building?
- What must the system remember?
- What changes state?
- What happens outside the request cycle?
- Where does AI fit?
- What will we cut if time becomes limited?

Week 4 Proposal: Required Repository Artifacts

Required core artifacts

- README.md
- docs/architecture.md
- submissions/week04/README.md

Strong supporting artifacts

- Schema diagram
- architecture flow diagram
- API draft
- state transition table
- risk register
- milestone timeline

Required checkpoint tag

- w04-proposal

Create and push it only after final review.



Start With One Primary User Workflow

Use this sentence pattern

- A **[primary user]** submits **[input]**.
The system validates and stores it.
A worker performs **[long-running or AI-enabled work]**.
The system stores **[result]**.
The user or reviewer retrieves **[status/output]**.

Rule

- Build one meaningful vertical slice before planning optional features.

Activity A: Define the Primary Workflow

Team activity — 10 minutes

Write your workflow as **6–8 numbered steps**.

Include:

1. user action;
2. API validation;
3. record creation;
4. job creation or async trigger;
5. worker action;
6. AI-enabled function;
7. persisted result;
8. status/result retrieval.

Then add one exception path:

- processing → failed → [retry / human review / visible error]

Identify Persistent Entities Before Drawing Tables

An entity usually deserves a table when it:

- is a meaningful thing the system must remember;
- has its own lifecycle or status;
- needs a stable ID;
- is referenced by another record;
- must be reviewed, queried, or audited later.

Typical initial target

- **4–7 meaningful entities**

Activity B: Draft Your Schema Diagram

Team activity — 10 minutes

For each proposed table, define:

Table	Primary key	Important fields	Foreign keys	Status/lifecycle

Design checks

- One fact in one appropriate place
- No vague “everything JSON” table
- Stable identifiers
- Required timestamps
- Clear status meanings
- Relationships match real workflow

Actor	Record	Allowed transition
API	Main record	draft → queued
Worker	Job	pending → claimed → completed/failed
Worker	Main record	queued → completed/failed
Reviewer	Review decision	pending → approved/rejected

Define Who Can Change State

State-transition responsibility table

Ask

- What is true now?
- What may happen next?
- Who is allowed to change it?
- What evidence should remain afterward?



AI Traceability and Human Review

For AI-enabled processing, consider persisting

- model, rules, or prompt/template version;
- input reference or input fingerprint;
- output label, score, summary, or recommendation;
- processing timestamp;
- reviewer decision or override reason, if applicable.

Also decide

- What is the fallback if AI processing fails?
- When should a result require human review?
- What sensitive information should not be stored or logged?



Architecture Is a Connected Data Story

For every component, explain:

- responsibility;
- data read/written;
- trigger;
- failure behavior;
- scope boundary.

User → API / validation → writesDatabase → reads and updatesWorker → callsAI-enabled function → writesPersisted result → API status/result endpoint

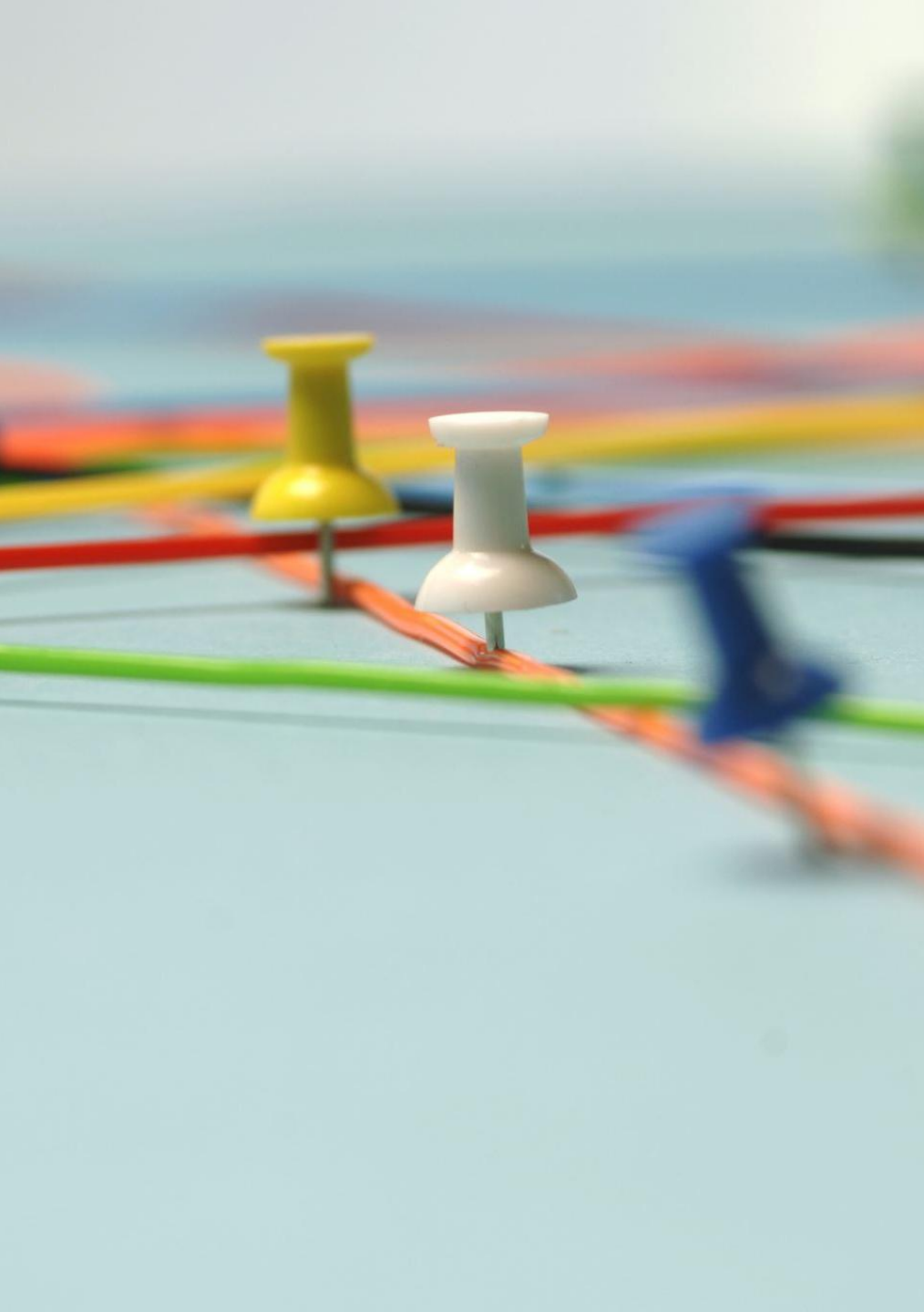
Milestone	Minimum credible outcome
Week 5	API contract and first persistence path
Week 7	End-to-end vertical slice
Week 9	Reproducible Docker deployment
Week 11	Reliability/observability/performance improvement
Week 12	Tested final workflow and demonstration

Milestones, Risks, and Scope Control

Plan for the semester, not an imaginary product

Name your first scope cut now

- Examples: dashboard, extra user roles, advanced model, external integration, multiple workflows.



Activity C: Build the Proposal Artifacts



Team studio — 15 minutes

Begin editing the real repository artifacts.

`docs/architecture.md`

- problem and primary user;
- primary workflow;
- architecture diagram;
- persistent entities and schema;
- state transitions;
- worker and AI plan;
- risks and scope cuts.

`submissions/week04/README.md`

- concise proposal summary;
- milestone plan;
- artifact index;
- AI Use Statement, if applicable.

Peer Review / Design Check



Swap documents with another team — 5 minutes

- Review against these questions:
- Can we identify one clear user workflow?
- Do the tables match the workflow?
- Are key relationships and statuses understandable?
- Is the worker/AI path explicit?
- Is the plan buildable by Week 7?
- Is there a credible scope cut?

Give:

- **one strength**
- **one ambiguity**
- **one risk or recommendation**

Before You Leave: Assign the Next Steps

Team exit checklist

- ✓ Architecture document has a first draft
- ✓ Week 4 submission README exists or is started
- ✓ Owner assigned for each unfinished section
- ✓ One person checks diagrams and relationships
- ✓ One person checks milestones and scope risks
- ✓ Team agrees on review and finalization time
- ✓ No tag has been created prematurely



Closing: Coherent and Buildable Beats Ambitious and Vague

Remember

Clear workflow

+ explicit data model

+ controlled state changes

+ realistic worker and AI path

+ documented risks and scope cuts

= credible engineering plan

Before submission:

```
git status
```

```
git add ...
```

```
git commit -m "Complete Week 4 architecture proposal"
```

```
git push
```

```
git tag -a w04-proposal -m "Week 4 proposal checkpoint"
```

```
git push origin w04-proposal
```